Laboratorio 2 | Pipeline seguro paso a paso

Cloud Forge · Módulo 3 · Sesión 2 | Material docente

1. Objetivo, entregables y seguridad

Construir CI con Ruff, pytest, Bandit y Trivy; provocar cinco fallos de forma independiente y demostrar que ninguno permite empaquetar. Trabajar en repositorio de práctica, sin credenciales reales y sin datos personales. El token sintético no permite acceso a ningún servicio.

Entregar enlace al repositorio, hash de cada commit, tabla de experimentos, reportes y ejecución final. Conservar archivos de código, workflow y configuración. Las salidas de esta guía son ilustrativas, no ejecuciones verificadas en un runner.

2. Requisitos y estructura

Cuenta GitHub, permisos para crear repositorios y ejecutar Actions, navegador, editor y Git. Para pruebas locales: Python 3.11; Docker opcional para Trivy. Se recomienda usar GitHub Actions para que todos compartan el mismo entorno. No se necesitan tokens propios, secretos de Actions ni servidor de despliegue.

```
app/__init__.py
app/calculadora.py
tests/test_calculadora.py
requirements.txt
requirements-dev.txt
pyproject.toml
.gitignore
security/trivy-secret.yaml
ci/check.sh
.github/workflows/secure-ci.yml
```

El proyecto inicial es correcto. requirements.txt no declara paquetes porque la aplicación usa la biblioteca estándar. requirements-dev.txt contiene solo herramientas didácticas; se excluye explícitamente del SCA de este ejercicio. No trasladar esa exclusión sin análisis a producción.

3. Crear y subir el repositorio

1. Descomprimir el proyecto y abrir la carpeta que contiene app, tests y .github.
2. En GitHub: New repository → nombre cloud-forge-ci → Create repository. Dejar sin README inicial si se usará Git. Elegir visibilidad según política institucional.
3. Abrir terminal en la carpeta del proyecto y ejecutar los comandos siguientes, sustituyendo la dirección por la que muestra GitHub. Si solicita autenticación, usar el método seguro configurado en el equipo; no pegar credenciales en archivos.

```
git init
git branch -M main
git add .
git commit -m "base: pipeline seguro"
git remote add origin DIRECCION_DEL_REPOSITORIO
git push -u origin main
```

Salida esperada: commit creado, rama main publicada y árbol del repositorio visible, incluida .github/workflows/secure-ci.yml. Alternativa gráfica: usar GitHub Desktop → Add local repository → Publish repository. Verificar que los archivos ocultos no hayan sido omitidos.

4. Revisar la aplicación y las pruebas

Abrir app/calculadora.py y tests/test_calculadora.py. Identificar dos funciones y tres pruebas. La conversión acepta un entero, pero rechaza expresiones. No se evalúa código recibido de usuarios.

```
def sumar(a: int, b: int) -&gt; int:
    return a + b
```

```
def interpretar_entero(texto: str) -&gt; int:
    return int(texto)
```

```
import pytest

from app.calculadora import interpretar_entero, sumar


def test_suma():
    assert sumar(2, 3) == 5


def test_entero():
    assert interpretar_entero("12") == 12


def test_rechaza_expresiones():
    with pytest.raises(ValueError):
        interpretar_entero("2 + 3")
```

Opcional local: crear entorno con python -m venv .venv; activar con source .venv/bin/activate en Linux/macOS o .venv\Scripts\Activate.ps1 en PowerShell; instalar con python -m pip install -r requirements-dev.txt. Ejecutar python -m pytest -q. Salida esperada: 3 passed, con tiempo variable.

5. Entender el workflow completo

Abrir .github/workflows/secure-ci.yml. Confirmar cinco valores de matriz, permisos de lectura, instalación de herramientas, timeout y conservación de informes. checks crea cinco trabajos; no es una única secuencia que se detiene en el primer error. package depende de todos ellos.

```
name: Secure CI
on:
  push:
    branches: [main]
  pull_request:
  workflow_dispatch:
permissions:
  contents: read
jobs:
  checks:
    name: control-${{ matrix.control }}
    runs-on: ubuntu-latest
    timeout-minutes: 15
    strategy:
      fail-fast: false
      matrix:
        control: [lint, tests, sast, secrets, sca]
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: '3.11'
      - name: Instalar herramientas
        run: python -m pip install -r requirements-dev.txt
      - name: Ejecutar control
        run: bash ci/check.sh "${{ matrix.control }}"
      - name: Conservar informe incluso si falla
        if: always()
        uses: actions/upload-artifact@v4
        with:
          name: reporte-${{ matrix.control }}
          path: reports/
          if-no-files-found: warn
          retention-days: 7
  package:
    needs: checks
    runs-on: ubuntu-latest
    timeout-minutes: 5
    steps:
      - uses: actions/checkout@v4
      - name: Empaquetar codigo aprobado
        run: |
          mkdir -p dist
          zip -r dist/cloud-forge.zip app requirements.txt
      - uses: actions/upload-artifact@v4
        with:
          name: codigo-aprobado
          path: dist/cloud-forge.zip
```

```
          retention-days: 7
```

Salida esperada: GitHub reconoce Secure CI en Actions. fail-fast: false mantiene los otros controles si uno falla. No añadir continue-on-error. if: always() aplica a la subida de reportes, no a la construcción ni a la aceptación de un hallazgo.

6. Revisar los comandos de cada control

```
#!/usr/bin/env bash
set -euo pipefail
mkdir -p reports
control="${1:?Indique lint, tests, sast, secrets o sca}"
case "$control" in
  lint) python -m ruff check app tests --output-format concise &gt; reports/lint.txt 2&gt;&amp;1 ;;
  tests) python -m pytest -q --junitxml=reports/tests.xml &gt; reports/tests.txt 2&gt;&amp;1 ;;
  sast) python -m bandit -r app -f txt -o reports/sast.txt ;;
  secrets|sca)
    scanners=secret
    extra=(--secret-config security/trivy-secret.yaml)
    if [[ "$control" == sca ]]; then
      scanners=vuln
      extra=(--severity HIGH,CRITICAL)
    fi
    docker run --rm -v "$PWD:/work" -w /work       aquasec/trivy:0.61.1 fs --scanners "$scanners"
       --exit-code 1 --skip-dirs .git,.venv,reports,dist       --skip-files requirements-dev.txt
       "${extra[@]}" --format table --output "reports/$control.txt" .
    ;;
  *) echo "Control no reconocido"; exit 2 ;;
esac
```

Salida esperada de un control limpio: código de proceso 0. Hallazgo configurado para bloquear: código distinto de 0. Trivy necesita --exit-code 1 para convertir hallazgos en fallo. El SCA filtra HIGH y CRITICAL; secretos no usa ese filtro. Se omiten .git y reportes para no confundir el árbol actual con el historial o con informes anteriores. Un error de descarga también bloquea, pero se diagnostica como fallo operativo, no como vulnerabilidad.

7. Ejecutar y documentar la línea base

1. Entrar a Actions → Secure CI → ejecución del commit inicial.
2. Si no aparece, confirmar rama main y ruta del YAML; se puede usar Run workflow.
3. Abrir cada control y descargar sus reportes.
4. Verificar package y descargar codigo-aprobado.

```
control-lint       success
control-tests      success
control-sast       success
control-secrets    success
control-sca        success
package            success
```

Contenido esperado: lint sin infracciones; pytest con 3 pruebas aprobadas; Bandit sin incidencias; Trivy sin hallazgos bloqueantes en la aplicación mínima. Es posible que el escáner indique ausencia de dependencias: es coherente con el manifiesto vacío, no demuestra cobertura de paquetes de desarrollo. Guardar captura con commit y estado, además de reportes.

8. Experimento 1: lint

1. Añadir import os al inicio de app/calculadora.py sin usarlo.
2. Guardar, confirmar y publicar.
3. Abrir control-lint y reporte-lint.
4. Registrar F401 y la ubicación de la importación.
5. Eliminar import os, confirmar y publicar de nuevo.

```
git add app/calculadora.py
git commit -m "demo: fallo lint"
git push
```

Salida esperada durante el fallo: F401 os imported but unused; control-lint fallido y package omitido. Los otros controles deberían aprobar. Tras corregir: lint verde y empaquetado habilitado. No corregir añadiendo # noqa: se elimina la causa.

9. Experimento 2: pruebas

Con la línea base restaurada: cambiar solo return a + b por return a - b. Confirmar y publicar. Leer control-tests, tests.txt y tests.xml. Después restaurar return a + b y publicar la corrección.

```
FAILED tests/test_calculadora.py::test_suma
AssertionError: assert -1 == 5
1 failed, 2 passed
```

Salida esperada: pruebas fallidas y package omitido. Ruff y Bandit pueden aprobar porque una resta es código sintácticamente correcto. Corregir la implementación, no cambiar el resultado esperado para ocultar el error.

10. Experimento 3: SAST

Con el repositorio nuevamente correcto: añadir la función siguiente al final de app/calculadora.py. El ejemplo usa una constante, no entrada externa; no usar eval con datos ajenos. Confirmar y publicar. Revisar el reporte SAST. Después eliminar la función completa y publicar la corrección.

```
def ejemplo_inseguro():
    return eval("2 + 3")
```

Salida esperada: Bandit identifica B307, uso de eval, severidad media; control-sast falla y package se omite. Las tres pruebas pueden continuar verdes porque no ejercitan esa función. La política didáctica de Bandit bloquea cualquier hallazgo: no se limita a HIGH/CRITICAL. No usar # nosec como solución.

11. Experimento 4: secreto sintético

1. Revisar la regla personalizada en security/trivy-secret.yaml.
2. Crear app/demo.env con la línea mostrada a continuación. El valor es ficticio y carece de permisos reales.
3. Confirmar y publicar; abrir control-secrets.
4. Eliminar app/demo.env; confirmar y publicar. No guardar el token completo en el informe de entrega si no es necesario.

```
rules:
  - id: cloud-forge-demo
    category: general
    title: Token sintetico de laboratorio
    severity: HIGH
    regex: 'CFDEMO_[A-Z0-9]{24}'
    keywords:
      - CFDEMO_

DEMO_TOKEN=CFDEMO_ABCDEFGHIJKLMNOPQRSTUVWX
```

Salida esperada: hallazgo HIGH con identificador cloud-forge-demo en app/demo.env; control-secrets falla y package se omite. La máscara exacta del valor puede variar. Tras eliminar el archivo: el árbol actual vuelve a aprobar, aunque la cadena sintética permanece en commits anteriores. Una credencial real exigiría revocación/rotación e investigación además de remediar repositorio e historial. Esta regla no detecta cualquier secreto posible.

12. Experimento 5: SCA

1. Restaurar el estado correcto del experimento anterior.
2. Añadir requests==2.19.1 a requirements.txt. Solo se modifica el manifiesto: no instalar ese paquete.
3. Confirmar y publicar.
4. Revisar control-sca y registrar al menos un hallazgo HIGH/CRITICAL, identificador y versión corregida sugerida.

5. Como la aplicación no usa Requests, eliminar la dependencia innecesaria y dejar el comentario inicial. Confirmar y publicar.

```
# Dependencia obsoleta solo para escaneo; no instalar.
requests==2.19.1
```

Salida esperada: Trivy reconoce Requests en el manifiesto y, con una base que contenga los avisos correspondientes, devuelve hallazgos bloqueantes; package queda omitido. No fijar una cantidad exacta de CVE. Si no se detecta nada: comprobar nombre requirements.txt, versión literal, actualización de base, logs y cobertura del analizador; no forzar un fallo artificial ni declarar detección sin evidencia.

La eliminación es válida porque el código no usa el componente. Si una aplicación sí lo necesita, elegir una versión corregida según avisos vigentes, actualizar el lock/inventario completo y repetir pruebas y SCA. Un requirements.txt con dependencias directas no representa por sí solo todas las transitivas.

13. Verificación final

Comprobar que no quedan import os, resta defectuosa, función con eval, demo.env ni Requests antiguo. Publicar un commit final si hubo cambios pendientes. Revisar que los cinco controles y package estén verdes. Descargar codigo-aprobado y verificar que contiene app/ y requirements.txt, pero no informes ni credenciales. El artefacto es un ZIP de código aprobado por estas reglas; no es una imagen ni prueba de firma o de procedencia.

14. Matriz de evidencias y entrega

Para cada experimento diligenciar: commit con fallo; control esperado; control observado; regla o aviso; archivo/línea o paquete; estado de package; corrección; commit corregido; enlace a ejecución; nombre del reporte. El paquete incluye una plantilla CSV. Adjuntar línea base y verificación final. No entregar únicamente capturas verdes.

15. Problemas frecuentes

Workflow no aparece: ruta, extensión YAML, indentación, rama y Actions habilitado.
Permission denied al script: usar bash ci/check.sh como indica el workflow.
ModuleNotFoundError: usar python -m pytest desde raíz y verificar app/__init__.py.
Cannot connect to Docker: verificar runner alojado; localmente iniciar Docker.
Descarga fallida de PyPI/Trivy/DB: revisar conexión y reintentar sin anular controles.
No hay reporte: revisar si el fallo ocurrió antes de crearlo; consultar log de instalación y paso fallido.
Escaneo no detecta token: revisar regla, archivo rastreado y los 24 caracteres.
SCA verde inesperado: revisar manifiesto, cobertura y base; un escaneo incompleto no es éxito.
Una nueva vulnerabilidad rompe línea base: revisar aviso y dependencia afectada; no añadir exclusiones amplias.

16. Preguntas y rúbrica

1. ¿Por qué una prueba puede aprobar y SAST fallar?
2. ¿Qué implica package omitido?
3. ¿Qué diferencia un secreto borrado de uno revocado?
4. ¿Qué cobertura falta al escanear solo dependencias directas?
5. ¿Por qué HIGH/CRITICAL no expresa por sí solo explotabilidad?
6. ¿Qué revisar antes de aceptar código generado por IA?

Rúbrica: línea base 15; cinco controles 40; bloqueo de empaquetado 15; correcciones y verificación 15; evidencia y reflexión 15. Para los 8 puntos de cada control: reproducir 2, interpretar 2, corregir 2, evidenciar 2. No se considera completo si se desactiva una puerta obligatoria.

17. Fuentes y límites

Fuentes: documento 03_Sesion2_Presentacion.pdf, 5 páginas. GitHub: https://docs.github.com/es/actions/tutorials/build-and-test-code/python . Trivy: https://trivy.dev/docs/latest/configuration/others/ ; https://trivy.dev/docs/latest/scanner/secret/ ; https://trivy.dev/docs/v0.51/guide/coverage/language/python/ . Bandit: https://bandit.readthedocs.io/en/latest/blacklists/blacklist_calls.html .

La guía amplía el documento original con un proyecto didáctico. No implementa control de licencias, escaneo de historial, escaneo de imágenes, SBOM, firma, procedencia ni publicación a un registro. La exclusión del manifiesto de herramientas solo acota el ejercicio. No se ha validado una ejecución remota desde este entorno; ensayar antes de orientar la clase.